

8일 차

이미지 모델링 기초 CNN

이번 차시에서 주목할 키워드

- 합성곱층
- 커널(필터)
- 특징 맵
- 맥스 풀링
- 풀치기
- 패딩

8일 차 학습 목표

- CNN의 기본 개념을 이해하고 이미지에서 특징 추출 원리를 설명할 수 있습니다.
- 합성곱층, 맥스 풀링층, 풀치기, 완전 연결층 등 CNN의 주요 요소를 살펴봅니다.
- MNIST와 CIFAR-10 데이터로 여러 CNN 모델을 구현하고 성능의 차이를 확인합니다.

이미지 모델링은 컴퓨터가 이미지를 인식하고 구별할 수 있도록 만드는 기술입니다. 예를 들어 손글씨 숫자를 구분하거나 사람의 얼굴을 식별하는 작업이 대표적인 사례입니다. 딥러닝에서는 이 과정을 자동화하기 위해 주로 합성곱 신경망(Convolutional Neural Network, 이하 CNN)을 사용합니다. CNN은 이미지의 중요한 특징(모서리, 패턴, 색상 등)을 스스로 학습하므로 사람이 직접 규칙을 만들지 않아도 높은 정확도로 이미지를 인식하고 분류할 수 있습니다.

CNN 개념 이해

모델이 이미지를 이해하는 법

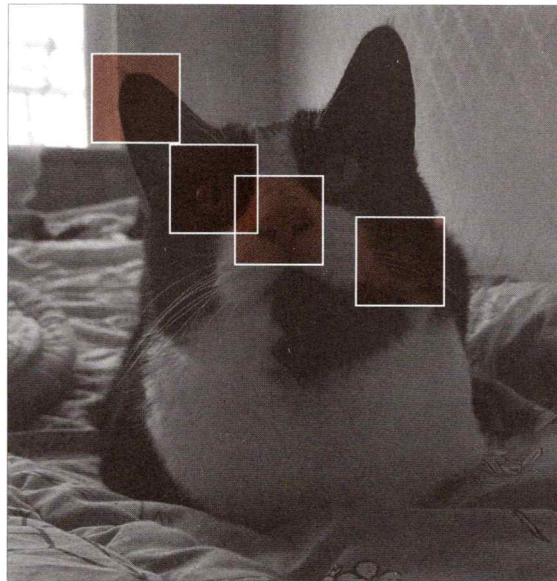
사람은 고양이 사진을 보면 특별히 노력하지 않아도 고양이라는 것을 직관적으로 알 수 있습니다. 왜 그럴까요? 사람이 영상 혹은 이미지를 인식하는 방법과 모델이 인식하는 방법을 비교하며 설명하겠습니다.

영역에서 특징 찾기

사람은 스스로 인지하지 못하지만, 뇌는 이미 수많은 경험을 통해 고양이의 특징을 구별할 수 있도록 학습되었습니다. 예를 들어 뽀족한 귀, 동그란 눈, 삼각형 모양의 코, 부드러운 털 같은 특징을 보면 사람은 이 동물이 고양이임을 인식합니다. 사람은 무언가를 볼 때 단번에 전체를 인식하는 것처럼 느껴지지만, 사실은 눈으로 들어온 다양한 부분 정보를 뇌가 종합하면서 이미지를 이해합니다.

CNN(Convolutional Neural Network)은 사람의 시각적인 인지 과정을 모방한 딥러닝 모델입니다. CNN도 사람의 인지 과정과 유사하게 이미지를 한 번에 파악하는 것이 아니라, 작은 영역 단위로 나누고 각각의 영역에서 중요한 특징들을 찾아냅니다. 그리고 이 특징들을 층층이 쌓아가며 전체 이미지를 점진적으로 이해합니다.

각각의 영역에서 추출된 특징들은 숫자로 표현되는데, 숫자는 해당 영역이 그 특



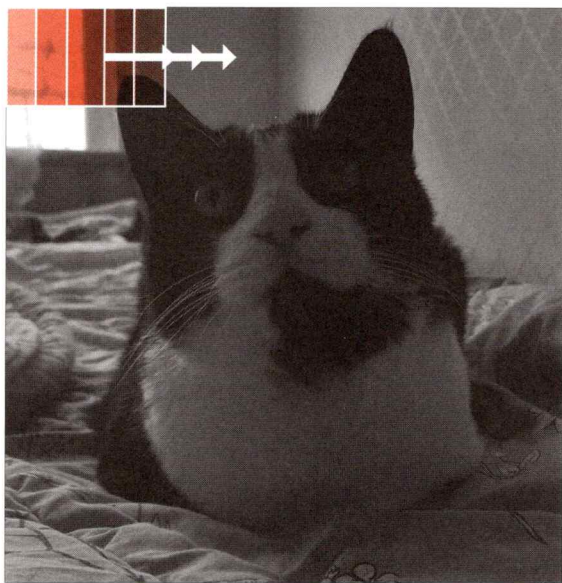
[그림 8-1] 고양이의 부분 특징

징을 얼마나 잘 나타내고 있는지를 의미합니다. 수치화된 정보를 바탕으로 모델은 “이 부분에 귀가 있다” 또는 “이 영역에는 눈의 특징이 없다” 같은 판단을 내리면서 이미지를 이해합니다.

특징을 찾는 도구: 필터

그렇다면 CNN은 특징들을 어떻게 찾을까요? 여기서 말하는 특징이란 이미지 속에서 특정 대상을 구분하는 데 도움이 되는 시각적인 단서입니다. 예를 들어 고양이의 뾰족한 귀, 사람의 눈동자, 숫자 ‘8’의 동그란 형태 등이 모두 특징에 해당합니다.

그렇다면 CNN은 이런 특징들을 어떻게 찾아낼까요? 특징을 찾을 때 중요한 역할을 하는 것이 바로 필터(Filter)입니다. 필터는 이미지 위를 일정한 간격으로 이동하면서 각각의 영역에 특정 패턴이 얼마나 존재하는지를 수치로 표현합니다. 한 가지 예로 고양이의 귀 모양을 인식하는 필터가 있다고 가정해 봅시다. [그림 8-2]처럼 필터는 이미지의 각 영역을 스캔하면서 귀 모양이 있는지 확인합니다. 그리고 해당 특징이 잘 드러나는 부분에는 높은 값을, 그렇지 않은 부분에는 0에 가까운 값을 출력합니다.



[그림 8-2] 부분 특징을 찾기 위한 필터의 이동

[그림 8-2]를 보면 필터는 고양이 이미지 왼쪽 위 모서리에서 시작해 오른쪽으로 조금씩 이동하면서 특징을 찾습니다. 오른쪽 끝에 도달하면 다시 아래로 이동해 처음부터 같은 과정을 반복합니다. 이와 같이 필터는 이미지를 전체적으로 스캔하면서 패턴을 찾습니다.

CNN 학습: 필터를 만드는 과정

CNN은 이미지를 분석하기 위해 필터(커널이라고도 함)를 사용합니다. 하지만 처음부터 어떤 필터가 어떤 특징을 찾아야 하는지는 알지 못합니다. 학습 초기의 필터 값은 무작위로 설정되어 있어 그 자체로는 아무런 의미가 없는 숫자에 불과합니다.

그러나 모델은 수많은 이미지를 반복적으로 학습하면서 어떤 패턴이 중요한지 점차 배웁니다. 예를 들어 고양이의 이미지를 학습할 때 처음에는 귀, 눈, 코 같은 부위를 구별하지 못하지만, 학습이 진행될수록 ‘이런 모양이 귀를 나타내는구나’, ‘이런 곡선은 눈의 형태구나’와 같은 특징을 스스로 찾아냅니다. 이처럼 학습이 진행되면 숫자로 표현된 이미지 데이터가 점차 의미를 갖게 되고 모델은 이미지의 내용을 인식할 수 있게 됩니다.

학습이 반복될수록 필터 값은 조정되며 모델은 특정 부위의 특징을 더욱 정확하게 감지할 수 있습니다. 초기 단계에서는 직선, 모서리, 색의 대비처럼 단순한 형태를 인식하지만, 층이 깊어질수록 이러한 기본 특징들이 결합되어 귀, 눈, 얼굴 윤곽과 같은 복잡하고 추상적인 패턴을 포착합니다. 이 과정을 통해 CNN은 점차 특징 추출(Feature Extraction) 능력을 발전시키며, 결국 이미지를 보다 높은 수준에서 이해하고 정확하게 분류할 수 있는 모델로 성장합니다.

CNN의 최종 목표: 예측

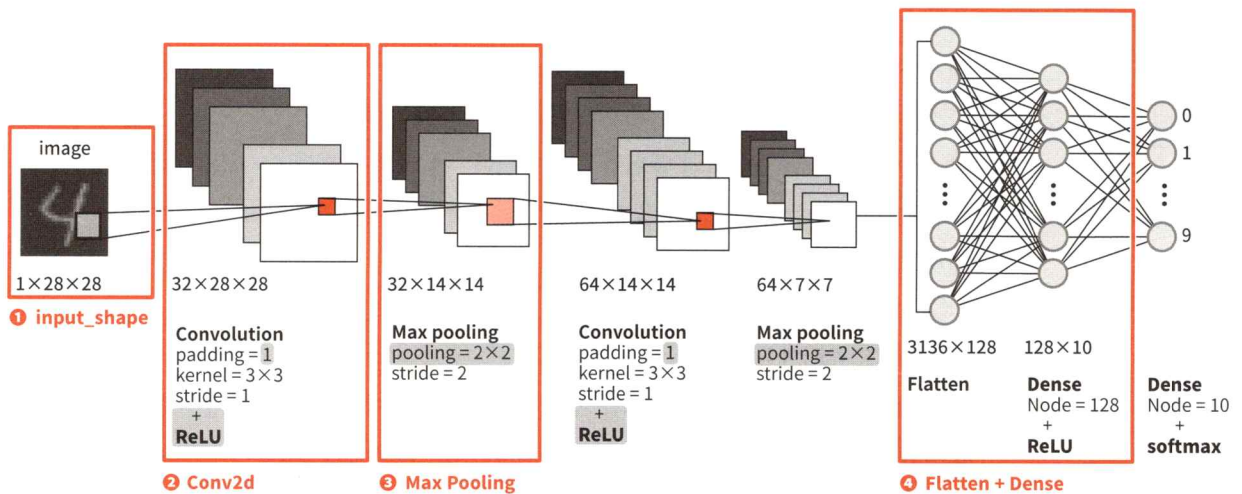
CNN이 만든 필터들은 단지 대상의 학습에만 쓰이는 것이 아니라, 새로운 이미지를 분석할 때도 그대로 활용됩니다. 예를 들어 한 번도 본 적 없는 고양이 사진이 주어지면 CNN은 이미 학습한 필터로 이미지 곳곳에서 귀, 눈, 얼굴 윤곽과 같은 특징을 찾습니다. 이 특징들이 과거 학습한 고양이의 패턴과 유사하다면 모델은 주어진 이미지를 고양이로 예측하고, 반대로 해당 특징이 잘 나타나지 않으면 고양이가 아니라고 판단합니다.

결국 CNN의 핵심 목표는 이미지를 영역별로 세세하게 나눈 다음, 각각의 영역에서 의미 있는 정보를 뽑아내고 이 정보를 기반으로 전체 이미지를 예측하는 일입니다. 특징 기반으로 접근하는 CNN은 사람이 사물을 인식하는 과정과 유사하기 때문에 이미지 분류뿐만 아니라 객체 탐지, 얼굴 인식, 자율주행 자동차의 시각 시스템 등 여러 응용 분야에서 매우 뛰어난 성능을 발휘하고 있습니다. CNN은 이미지를 단순히 보는 것을 넘어 본질적인 패턴까지 이해하는 매우 강력한 도구입니다.

이미지 모델링 절차

[그림 8-3]은 CNN(합성곱 신경망) 모델의 일반적인 구조입니다. 처음 이 구조를 볼 때는 복잡하게 느낄 수 있지만, 실제로는 네 가지 핵심 요소만 이해하면 됩니다. 다시 말해 이 네 요소만 이해하면 이미지를 모델 구조에 통과시키면서 처리하는 각각의 단계와 특징 추출 과정이 한눈에 보입니다.

이제 [그림 8-3]의 처리 절차와 구조를 참조하면서 각각의 단계가 어떻게 구성되어 있는지 살펴보겠습니다.



[그림 8-3] CNN 모델 구조 예시

1 입력(Input) 단계: 이미지의 형태 정의하기

CNN이 이미지를 이해하려면 먼저 '이 이미지가 어떤 크기와 형태를 가진 데이터'인지 명확히 정의해야 합니다. 마치 사람이 어떤 책을 읽기 전에 글자가 어떤 언어로 쓰여 있는지, 책의 크기와 페이지 수는 얼마인지 확인하는 과정과 비슷합니다. 모델은 색상 채널(Channel)과 높이(Height), 너비(Width)를 알아야 이미지 정보를 바르게 처리할 수 있습니다.

- 흑백 이미지는 색상 정보가 단일 채널이므로 [1, 높이, 너비] 형태로 표현됩니다. 예를 들어 [그림 8-3]에서 숫자 '4'가 적힌 28×28 픽셀 크기의 손글씨 이미지는 [1, 28, 28] 형태입니다. 여기서 '1'은 색상이 흑백이라는 뜻이고, 28×28 은 이미지가 세로와 가로 각각 28픽셀로 이루어졌다는 뜻입니다.
- 컬러 이미지라면 조금 다릅니다. 흔히 보는 사진은 빨강(R), 초록(G), 파랑(B) 세 가지 색상 정보가 합쳐져 표현되는데, CNN에서는 이를 각각 하나의 채널로 다룹니다. 따라서 컬러 이미지는 [3, 높이, 너비] 형태입니다. 예를 들어 64×64 픽셀의 컬러 사진은 [3, 64, 64]로 표현됩니다.

이 단계는 단순히 데이터의 모양만을 알려주는 것 같지만, 실제로는 모델로 하여금 이미지를 제대로 이해하도록 도와주는 중요한 과정입니다. CNN은 이 정보가 있어야 올바른 크기의 필터를 적용해 한 칸씩 이동하며 이미지를 분석할 수 있습니다.

입력 단계는 모델이 ‘받아들일 이미지가 어떤 형태인지’ 미리 인식하고 준비하는 과정이라고 볼 수 있습니다.

이미지의 크기와 채널 수는 모델의 성능과 연산 속도에 영향을 줍니다. 크기가 큰 이미지는 더 많은 정보를 담을 수 있지만, 처리 시간이 오래 걸립니다. 반대로 너무 작은 이미지는 중요한 정보를 잃을 수 있습니다. 따라서 CNN 모델을 설계할 때는 입력 크기를 적절히 조정하는 전처리 작업이 중요하며, 이는 모델 성능을 좌우하는 중요한 요소입니다.

② 합성곱 단계: 특징 추출하기

이미지가 CNN에 들어오면 가장 먼저 거치는 단계가 합성곱층(Convolutional Layer)입니다. 이 단계는 마치 탐정이 돋보기를 들고 사진을 구석구석 살펴보며 “여기에 중요한 단서가 있나?”라고 하며 특징을 찾는 과정과 비슷합니다. CNN은 이 일을 필터(Filter)라는 작은 도구를 사용해 수행합니다.

필터는 마치 작은 창을 통해 이미지를 보는 것처럼 이미지의 일부 영역(예: 3×3 픽셀)을 살피는 도구입니다. 모델은 관찰하려는 이미지에서 필터를 조금씩 이동시키면서(이를 스트라이드(Stride)라고 함) 각각의 영역에서 패턴을 찾습니다.

필터 과정에서 만들어지는 것이 바로 특징 맵(Feature Map)입니다. 특징 맵은 “이미지의 이 영역에서는 이런 패턴이 강하게 보인다”와 같은 정보를 숫자로 표현한 지도입니다. CNN은 처음에는 직선, 모서리 같은 단순한 특징을 찾는데, 합성곱층을 여러 겹 쌓을수록 점차 복잡한 패턴을 인식합니다. 결국 여러 층을 거친 CNN은 “이건 귀 모양 같다”, “이건 숫자 4의 윗부분 같다”라고 말하는 것처럼 더욱 구체적이고 추상적인 특징을 추출하기 시작합니다.

또한 합성곱 연산 후에는 ReLU(활성화 함수)를 사용해 불필요한 음수값을 제거하고 중요한 특징만 남깁니다. 이 과정을 거치면 모델은 복잡한 패턴을 더 잘 학습할 수 있어 단순한 숫자 계산이 아니라 이미지의 의미까지 파악할 수 있습니다.

즉, 합성곱 단계는 CNN이 이미지를 한 조각씩 잘라 살펴보면서 그 속에서 의미 있는 패턴을 찾는 단계입니다. 이 단계가 잘 작동해야 이후 과정에서 “이 이미지는 무엇이다”라고 정확히 예측할 수 있습니다.

③ 맥스 풀링(Max Pooling) 단계: 중요한 정보만 골라내기

합성곱 단계를 거치면 이미지는 여러 개의 특징 맵(Feature Map)으로 변환됩니다. 그런데 특징 맵을 그대로 사용하면 데이터가 너무 크고 복잡해 계산량이 많아집니다. 그럼 모델이 불필요한 세부 정보까지 신경 쓰게 됩니다. 이럴 때 풀링(Pooling)이라는 과정을 거치면 꼭 필요한 정보만 골라내고 나머지는 과감히 줄입니다. 마치 사진을 멀리서 볼 때는 큰 형태만 기억하고 세세한 점이나 잡음 등은 무시하는 것과 비슷합니다.

풀링에는 여러 방식이 있지만, 가장 많이 사용하는 방법이 맥스 풀링(Max Pooling)입니다. 이름에서 알 수 있듯이 작은 영역(보통 2×2 크기)의 값 중에서 가장 큰 값 하나만 남기고 나머지는 버리는 과정입니다. 풀링 단계를 거치면 이미지는 조금 더 작아지는 대신 가장 강하게 나타나는 특징(패턴)은 그대로 유지됩니다. 예를 들어 숫자 '4'의 이미지라면 교차하는 선과 같이 가장 강하게 보이는 지점은 남기고 주변의 희미한 흔적은 사라집니다.

풀링 단계를 거치면 이미지 데이터의 크기가 줄어들어 계산 속도가 빨라지는 것은 물론, 모델이 중요한 특징에 집중하는 장점이 있습니다. 또한 풀링은 이미지가 약간 회전했거나 위치가 바뀌어도 비슷한 특징으로 인식하도록 해주는 효과도 있습니다. 따라서 모델이 좀 더 안정적이고 유연하게 학습할 수 있도록 도와줍니다.

합성곱 연산을 수행하는 층이 합성곱층이고, 맥스 풀링 연산을 수행하는 층이 바로 맥스 풀링층입니다. 두 층은 마치 한 팀을 이루듯이 이미지를 분석합니다. 합성곱층이 돋보기로 세세한 단서를 찾는다면 맥스 풀링층은 이 단서에서 정말 중요한 부분만 메모하는 비서 같은 역할을 합니다. 이 두 과정을 반복하면 모델은 점점 더 효율적으로 이미지를 이해할 수 있습니다.

④ 완전 연결층과 분류 단계: 최종 결정 내리기

CNN 모델의 전반부에서는 이미지를 작은 영역으로 나누어 합성곱과 풀링으로 각각의 영역에서 중요한 특징을 찾습니다. 이제 남은 단계는 이 모든 특징을 하나로 모아 최종 결정을 내리는 일입니다. 이 역할을 하는 층이 바로 완전 연결층(Dense Layer)입니다.

완전 연결층은 말 그대로 모든 노드가 앞 단계의 특징과 모두 연결되는 층입니다. 앞 단계에서 여러 층을 거치며 추출한 특징들은 아직 조각난 이미지의 단서일

뿐입니다. 완전 연결층은 이 단서들을 모아 “이 이미지는 고양이다” 혹은 “이 이미지는 자동차다”와 같은 최종 결론을 내립니다. 마치 탐정이 사건의 여러 증거를 모아 전체 사건을 재구성하듯이 모델도 조각난 특징들을 하나로 묶어 결론을 내립니다.

이 과정을 위해 먼저 평탄화(Flatten) 작업을 거쳐 데이터를 1차원으로 변환합니다. 합성곱과 풀링 연산을 거친 후의 데이터는 2차원 혹은 3차원 형태로 되어 있으므로 이를 1차원 벡터로 펼쳐 완전 연결층에 넣어야 합니다. 완전 연결층에 넣으면 특징들은 쭉 일렬로 나열되면서 완전 연결층의 각 노드와 연결됩니다.

마지막으로 모델은 출력층(Output Layer)에서 예측을 수행합니다. 예를 들어 숫자 이미지를 0~9까지 분류하는 문제라면 출력층에는 10개의 노드로부터 10개의 로짓 값이 만들어집니다. 그리고 소프트맥스(Softmax) 함수를 사용해 0~9로 분류될 확률을 계산합니다. 그중 가장 높은 확률을 가진 숫자가 모델의 최종 예측 결과입니다.

완전 연결층 단계는 지금까지 학습한 수많은 특징을 한곳에 모아 종합 판단을 내리는 단계입니다. 합성곱이 ‘세부 단서 찾기 전문가’라면 완전 연결층은 ‘최종 결론을 내려주는 판사’와 같습니다. 이 두 단계가 함께 작동하면서 CNN은 이미지를 효과적으로 분류합니다.

CNN 구조의 핵심 4단계를 다시 정리하면 다음과 같습니다.

- 입력(Input Shape) 단계: 이미지의 크기와 채널 수를 정의합니다(예: 28×28 픽셀, 1채널)
- 합성곱층(Convolutional Layer) 단계: 필터로 이미지의 지역적인 특징을 추출합니다.
- 맥스 풀링층(Max Pooling) 단계: 추출된 특징을 요약해 계산량을 줄입니다.
- 완전 연결층(Dense Layer) 단계: 추출된 특징을 종합해 최종적으로 이미지를 분류합니다.

이 4단계만 기억하면 CNN 모델의 전체 구조를 이해할 수 있습니다. CNN의 구조가 복잡해 보여도 각각의 단계가 어떤 역할을 수행하는지 알면 전체 흐름이 훨씬 명확해집니다.

실습 1: MNIST 데이터로 CNN 모델 만들기

이번 절에서는 5일 차 다중 분류 모델링에서 다뤘던 MNIST 데이터를 이용해 간단한 형태의 CNN(합성곱 신경망) 모델링을 구현하면서 주요 연산 과정을 설명합니다.

CNN으로 어떻게 이미지를 처리하고 학습하는지 파악한다면 모델의 작동 원리를 좀 더 구체적으로 이해할 수 있습니다. 실습에서는 파이토치의 `nn.Conv2d` 모듈을 중심으로 합성곱(Convolution), 스트라이드(Stride), 패딩(Padding) 등 CNN의 핵심 개념을 코드로 구현합니다.

CNN 모델링 빠르게 구현하기

간단한 형태의 CNN 모델링을 구현하면서 CNN의 주요 개념들을 이해합니다. 먼저 새 코랩 파일을 생성하고 파일 이름은 '8일차_이미지모델링실습1.ipynb'로 합니다.

환경 준비에서 데이터 전처리까지

코랩 파일의 상단 메뉴에서 [런타임] - [런타임 유형 변경] 메뉴를 선택한 다음, 나오는 [런타임 유형 변경] 대화상자에서 하드웨어 가속기로 'T4 GPU'를 선택합니다.

빈 셀에서 환경 준비를 위한 라이브러리 불러오기 → 디바이스 할당 → 사용자 정의 함수의 순서로 코드를 작성하고 실행합니다. 이 코드들은 5일 차 185~187쪽을 참고해 여러분이 직접 작성합니다.

이어서 MNIST 데이터를 불러온 다음, 학습 데이터의 데이터 로더와 검증 및 테스트 데이터를 각각 준비합니다. 빈 셀에서 다음과 같이 작성합니다.

CODE

```
from torchvision import datasets
from torchvision.transforms import ToTensor

# 데이터 다운로드
train_set = datasets.MNIST(root="data", train=True, download=True,
transform=ToTensor())
test_set = datasets.MNIST(root="data", train=False, download=True,
transform=ToTensor())

# 학습 데이터의 데이터 로더 구성
batch_size = 64
train_loader = DataLoader(train_set, batch_size = batch_size)

# 검증, 테스트 데이터 준비
```

```

## 데이터 분할
x_val, x_test = test_set.data[:5000], test_set.data[5000:]
y_val, y_test = test_set.targets[:5000], test_set.targets[5000:]

## 스케일링
x_val = x_val / 255
x_test = x_test / 255

## 이미지 처리를 위한 차원 맞추기
x_val = x_val.view(5000, 1, 28, 28)
x_test = x_test.view(5000, 1, 28, 28)

```

데이터 전처리 코드는 지금까지 여러 번 구현했기 때문에 주석으로도 충분히 이해할 수 있어 따로 언급하지는 않겠습니다. 잘 모르겠다면 5일 차를 다시 한번 참조하기 바랍니다.

CNN 모델 설계

이어서 간단한 모델을 설계합니다. CNN 모델링에서는 네 가지 구성 요소(합성곱층, 맥스 풀링층, 펼치기, 완전 연결층)를 정확히 이해하는 것이 중요합니다.

빈 셀에서 다음과 같이 코드를 작성합니다.

CODE

```

n_class = 10
model = nn.Sequential(
    nn.Conv2d(in_channels=1, out_channels=32, kernel_size=3, stride=1,
              padding=1), ①
    nn.ReLU(),
    nn.MaxPool2d(kernel_size=2, stride=2), ②
    nn.Flatten(), ③
    nn.Linear(32*14*14, 64),
    nn.ReLU(),
    nn.Linear(64, n_class) ④
).to(device)

loss_fn = nn.CrossEntropyLoss()
optimizer = Adam(model.parameters(), lr=0.001)

```

- ① 합성곱층: 이 층에서 필터는 입력된 이미지를 훑어 가며 특징을 찾습니다.
- ② 맥스 풀링층: 합성곱층의 결과를 요약/압축합니다.
- ③ 펼치기: 맥스 풀링층의 결과를 1차원으로 펼칩니다. nn.Linear로 연결하기 위함입니다.
- ④ 출력층: 0~9까지의 숫자를 분류합니다.

이 4가지 요소는 잠시 후 CNN 모델링의 주요 개념들을 소개하면서 보다 상세히 설명합니다. 지금은 CNN 모델의 구조와 흐름에 초점을 맞춰 코드를 작성하면 됩니다. 계속해서 학습 및 예측, 평가를 구현하겠습니다.

학습 및 예측 평가

학습 및 예측 평가 과정 역시 종전의 코드와 동일합니다. 다음과 같이 코드를 작성합니다.

CODE

```
# 학습
epochs = 10
tr_loss_list, val_loss_list = [], []
for t in range(epochs):
    tr_loss = train(train_loader, model, loss_fn, optimizer, device)
    val_loss, _ = evaluate(x_val, y_val, model, loss_fn, device)
    tr_loss_list.append(tr_loss)
    val_loss_list.append(val_loss)
    print(f"Epoch {t+1}, train loss : {tr_loss:.4f}, val loss : {val_loss:.4f}")

# 예측
_, pred = evaluate(x_test, y_test, model, loss_fn, device)
pred = nn.functional.softmax(pred, dim=1)
pred = np.argmax(pred.cpu().numpy(), axis = 1)

# 평가
print(classification_report(y_test.numpy(), pred, digits = 4))
```

OUTPUT

	precision	recall	f1-score	support
0	0.9866	0.9923	0.9895	520
1	0.9965	0.9965	0.9965	564
2	0.9842	0.9900	0.9871	502
3	0.9807	0.9941	0.9873	510
4	0.9938	0.9938	0.9938	482
5	0.9817	0.9862	0.9840	436
6	0.9861	0.9980	0.9920	496
7	0.9864	0.9826	0.9845	516
8	0.9937	0.9794	0.9865	485
9	0.9937	0.9693	0.9814	489
accuracy			0.9884	5000
macro avg	0.9883	0.9882	0.9882	5000
weighted avg	0.9884	0.9884	0.9884	5000

10번의 에포크로 모델을 학습시키고 테스트 데이터로 예측하고 평가했습니다.

classification_report 함수에서 사용한 digits=4 옵션은 예측 결과의 수치를 소수점 넷째 자리까지 표시합니다.

5일 차에서 단순히 nn.Linear만을 사용해 만든 모델의 정분류율(Accuracy)은 약 0.95 정도였습니다. 이번에 구성한 CNN 모델은 별도의 하이퍼파라미터 튜닝을 하지 않았음에도 약 0.98~0.99 수준의 정확도를 보여 줍니다. 아직 세부적인 조정을 거치지 않은 상태라 성능을 직접 비교하기는 조금 이르지만, MNIST 이미지를 단순 선형 모델로 처리했던 것보다 CNN을 활용한 모델이 훨씬 더 뛰어난 성능을 발휘한다는 점은 확인할 수 있습니다.

다음 소절부터는 지금까지 구현한 모델의 코드를 하나하나 상세히 설명합니다. 코드 구현 과정을 머릿속에 떠올리면서 CNN 모델의 중요 개념들을 한 단계 더 깊이 이해합니다.

합성곱 연산 이해

nn.Conv2d 함수

nn.Conv2d는 CNN에서 이미지를 처리할 때 사용하는 핵심 연산 함수입니다. 이 함수는 이미지의 작은 영역을 살펴보면서 중요한 특징(Local Feature)을 찾는 역할을 합니다. 사람이 사진을 볼 때 특정 부분(귀, 눈, 코 등)에 집중해 하나하나 살펴듯이 Conv2d는 필터(커널)를 이용해 이미지에서 의미 있는 패턴을 하나씩 찾습니다.

CODE

```
nn.Conv2d(in_channels=1, out_channels=32, kernel_size=3, stride=1, padding=1),  
nn.ReLU(),
```

코드에서 사용한 옵션은 다음과 같습니다.

- `in_channels=1`: 입력 이미지의 채널 수입니다. 흑백 이미지는 색상이 하나(채널 1)뿐이므로 1로 설정합니다. 컬러 이미지라면 R, G, B 3개의 채널이므로 `in_channels=3`이 됩니다.
- `out_channels=32`: 출력되는 채널 수로서 필터의 개수입니다. 32개의 서로 다른 필터를 사용해 이미지를 분석하므로 32개의 서로 다른 '특징 맵(Feature Map)'이 생성됩니다. 각각의 필터는 직선, 곡선, 모서리 등 서로 다른 패턴을 학습합니다.

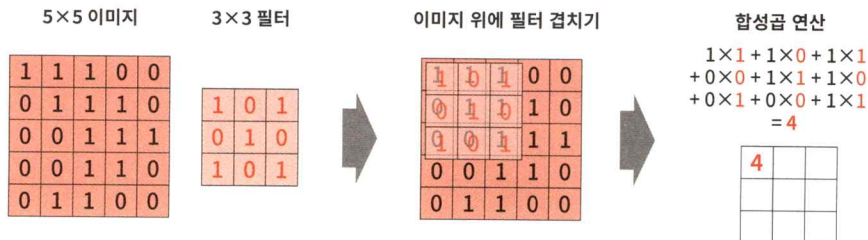
- `kernel_size=3`: 필터(커널)의 크기를 지정합니다. 여기서는 3×3 크기의 작은 창입니다. 이 창으로 이미지를 훑으며 패턴을 찾습니다. 크기가 크면 더 넓은 영역을, 작으면 더 세밀한 패턴을 찾을 수 있습니다.
- `stride=1`: 필터가 한 번에 이동하는 간격입니다. 1이면 한 칸씩 차례대로 이동하면서 이미지를 훑습니다.
- `padding=1`: 이미지의 가장자리를 처리할 때 0으로 채워 계산 범위를 확장하는 옵션입니다. 0으로 채우면 합성곱 연산 후에도 이미지의 가로×세로의 크기를 유지할 수 있습니다.

필터의 사용 과정을 비유하자면 돋보기를 들고 이미지를 한 부분씩 살펴보는 일과 같습니다. 돋보기의 크기가 `kernel_size`, 한 번에 움직이는 간격이 `stride`, 가장자리까지 보기 위해 빈 공간을 채우는 작업이 `padding`이라 생각하면 이해하기 쉽습니다.

합성곱 연산

합성곱은 작은 창(필터, 또는 커널)으로 이미지를 일정한 규칙에 따라 훑으면서 같은 위치의 숫자를 곱한 다음 모두 더하는 연산입니다. 이 필터 연산으로 이미지의 지역적인 패턴을 감지할 수 있습니다. 좀 더 쉽게 설명하면 필터(Filter)는 작은 도장처럼 이미지 위를 일정 간격으로 이동하며 특정 패턴을 찾습니다. 패턴을 찾는다는 합성곱 연산을 수행한다는 뜻입니다. 합성곱 연산은 이미지의 특정 부분과 필터를 겹치게 한 다음, 같은 위치의 값끼리 곱하고(element-wise multiply) 모두 더하는 계산입니다.

합성곱 연산을 어떻게 수행하는지 실제로 계산해 보겠습니다.



[그림 8-4] 합성곱 연산 예제(5×5 이미지에 3×3 필터 적용하기)

[그림 8-4]처럼 5×5 입력 이미지와 3×3 필터가 있다고 할 때(왼쪽 그림), 5×5 이미지 왼쪽 윗부분에 3×3 필터를 겹쳐 놓습니다(가운데 그림). 그리고 겹쳐진 부분에서 같은 위치(이미지와 필터)의 값끼리 곱하고 모두 더합니다(오른쪽 그림).

연산식은 다음과 같습니다.

$$1 \times 1 + 1 \times 0 + 1 \times 1 + 0 \times 0 + 1 \times 1 + 1 \times 0 + 0 \times 1 + 0 \times 0 + 1 \times 1 = 4$$

수식의 결과값 4는 필터가 특정 위치에서 합성곱 연산을 수행한 결과입니다. 그런 다음 스트라이드(stride)로 지정한 값만큼 옆으로 이동해 합성곱 연산을 수행하면서 나머지 결과값도 계산합니다. 스트라이드로 이동하면서 연산하면 이미지에서 패턴을 추출한(합성곱 연산을 수행한) 결과가 2차원 지도(Map)처럼 만들어집니다. 이 결과를 다른 표현으로 특징이 추출된 맵, 간단히 특징 맵(Feature Map)이라고 합니다. 합성곱층을 거쳐 생성된 특징 맵은 이미지의 각 영역에서 특정 패턴이 얼마나 잘 드러나는지를 수치로 표현한 것입니다. 이때 특정 영역이 필터가 찾으려는 패턴과 잘 맞을수록 합성곱 연산의 결과는 높게 나타납니다. 특징 맵은 활성화 함수(보통 ReLU)를 거쳐 다음 층으로 이동합니다.

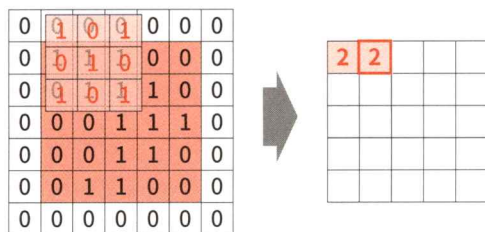
스트라이드: 필터가 움직이는 속도

스트라이드(Stride, 보폭)는 필터가 이미지를 얼마나 빨리 이동할지 결정하는 옵션입니다. 스트라이드는 기본값이 1입니다. 1로 지정하면 오른쪽으로 1칸 이동하고 오른쪽 끝까지 이동하면 아래로 1칸 이동한 다음, 다시 왼쪽 처음부터 오른쪽으로 이동합니다. 스트라이드를 2로 지정하면 오른쪽으로 2칸, 아래로 2칸씩 이동합니다. 스트라이드가 크면 합성곱 연산의 출력(특징 맵) 크기는 작아지는 반면, 연산 속도는 빨라집니다. 스트라이드가 작으면 더 많은 정보를 추출하므로 출력 크기는 커지지만, 연산 속도는 느려집니다.

패딩: 이미지 크기 유지하기

날씨가 추우면 패딩(Padding) 점퍼를 자주 입습니다. ‘패딩’이라는 말은 ‘덧댄다’는 뜻입니다. 합성곱 연산을 거치면 필터의 크기와 스트라이드 값에 따라 특징 맵의 크기가 줄어듭니다. 이를 막기 위해 패딩을 사용해 입력 이미지의 가장자리를 0으로 덧댑니다.

padding=1로 지정하면 이미지 주변에 1픽셀 두께로 0을 추가하는데, 0을 덧대고 합성곱 연산을 수행하면 입력 이미지의 크기를 연산 수행 후에도 그대로 유지할 수 있습니다. 패딩은 실제 업무에서 자주 사용하는데, 패딩을 적용하면 입력 이미지의 모서리 특징을 좀 더 잘 추출합니다.



[그림 8-5] 패딩을 적용한 합성곱 연산

ReLU 함수

nn.Conv2d 함수 다음에는 ReLU 함수가 옵니다. 합성곱 연산 결과는 활성화 함수 ReLU를 통과합니다. ReLU 함수는 모델로 하여금 더 복잡한 비선형 패턴을 학습하도록 합니다. 또한 ReLU 함수는 학습 연산 속도를 빠르게 할 뿐만 아니라, 기울기 소실 문제(Vanishing Gradient)를 완화합니다.

지금까지 살펴본 nn.Conv2d 함수의 핵심 요소들을 다시 정리하면 다음과 같습니다.

- 합성곱(Convolution) 연산: nn.Conv2d에서 필터로 이미지를 스캔해 지역적인 특징을 추출합니다.
- 스트라이드(Stride): 필터가 이동하는 간격을 조정합니다. 빠르게 이동할수록 연산량과 특징 맵의 크기는 줄어듭니다.
- 패딩(Padding): 이미지 가장자리에 0을 추가해 입력 이미지의 크기를 유지하고 정보 손실을 줄입니다.
- ReLU 활성화 함수: 비선형성을 추가해 모델의 표현력을 강화합니다.

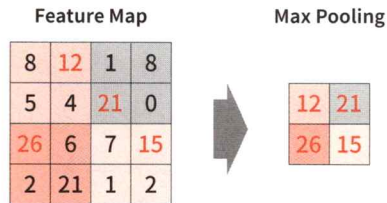
맥스 풀링 이해

맥스 풀링(Max Pooling)은 CNN에서 자주 사용하는 연산으로 입력된 특징 맵(Feature Map)을 더 작고 요약된 형태로 만드는 과정입니다. 일반적으로 '2×2 크기'의 작은 구역(창)을 설정한 다음, 이 창을 겹치지 않게 이동시키면서 각각의 구역에서

가장 큰 값을 하나씩 선택합니다. 맥스 풀링을 하면 이미지의 크기를 줄이면서도 해당 영역에서 가장 두드러진 특징을 유지할 수 있습니다.

맥스 풀링의 효과는 다음과 같습니다.

- 이미지의 중요한 특징(feature)은 유지하면서도 전체 크기를 줄일 수 있습니다.
- 이미지가 조금 이동하거나 왜곡되더라도 결과가 크게 달라지지 않기 때문에 더 견고하고 일반화된 모델을 만들 수 있습니다.
- 데이터의 크기가 작아지면 계산량은 줄고 학습 속도는 빨라집니다.



[그림 8-6] 맥스 풀링의 개념

[그림 8-6]을 보면 큰 입력 이미지를 2×2 구역으로 각각 나눈 다음, 각각의 구역에서 가장 큰 값만 선택해 새로운 출력 이미지를 만듭니다. 이런 식으로 맥스 풀링은 입력 이미지를 보다 간단한 형태로 압축하면서도 핵심 정보는 유지합니다.

다음은 맥스 풀링층의 코드입니다.

```
CODE
nn.MaxPool2d(kernel_size=2, stride=2, ...)
```

nn.MaxPool2d는 파이토치에서 맥스 풀링을 수행하는 함수입니다. kernel_size=2는 2×2 크기의 창(구역)을 의미합니다. stride=2로 창이 한 번에 2칸씩 이동하도록 설정합니다.

보통은 kernel_size와 stride를 동일한 크기로 지정합니다. 이렇게 하면 필터가 이미지를 훑을 때 한 칸씩 이동하면서 바로 옆 구역과 겹치지 않게 영역을 나눌 수 있습니다. 즉, 이미지 전체를 격자처럼 일정한 크기로 구분해 풀링하기 때문에 계산 결과가 명확하고 해석하기도 쉽습니다. 예를 들어 28×28 크기의 이미지에 kernel_size=2, stride=2 설정으로 맥스 풀링을 적용하면 출력은 절반 크기인 14×14가 됩니다. 출력 크기는 다음과 같은 공식으로 계산합니다.

$$28 // 2 = 14$$

// 연산자는 나눗셈에서 나머지는 버리고 몫만 취할 때 사용합니다. [그림 8-6]에 있는 특징 맵의 경우 세로와 가로 픽셀 수가 4개이고 필터의 크기는 2이므로 맥스 풀링 연산을 하면 $4 // 2 = 2$ 입니다. 4×4 크기의 특징 맵이 맥스 풀링층을 거치면 2×2 크기가 됩니다.

맥스 풀링이 필요한 이유를 좀 더 구체적으로 설명하면 다음과 같습니다. 합성곱 연산은 이미지 위를 필터가 이동하면서 각 영역에 얼마나 특정 패턴이 잘 나타나는지를 숫자로 표현합니다. 예를 들어 특정 필터가 '수직선' 모양을 감지하도록 학습되었다면 입력 이미지의 수직선에서는 큰 값이 나오고 그렇지 않은 영역에서는 작은 값이 나옵니다. 이렇게 생성된 특징 맵은 각각의 위치에 어떤 특징이 얼마나 강하게 나타나는지를 수치로 표현합니다. 맥스 풀링층은 특징 맵을 압축하면서도 각각의 구역에서 가장 뚜렷한 특징은 남기고 나머지는 제거합니다. 즉, 각각의 구역에서 가장 강한 패턴만 남겨 다음 단계로 전달하므로 모델이 불필요한 세부 정보에 신경 쓰지 않아 더 안정적으로 학습할 수 있습니다.

실습 2: MNIST 데이터로 CNN 모델 만들기 2(다중 합성곱층)

실습 1에서는 1개의 합성곱층과 1개의 맥스 풀링층으로 간단한 형태의 CNN 모델을 생성했습니다. 이번에는 여러 개의 합성곱층(Conv2D)과 맥스 풀링층(Max Pool2D)으로 더 깊은 구조의 CNN 모델을 만듭니다. 이번 실습은 앞에서 만들었던 '8일차_이미지모델링실습1.ipynb' 파일에서 이어 진행하면 됩니다.

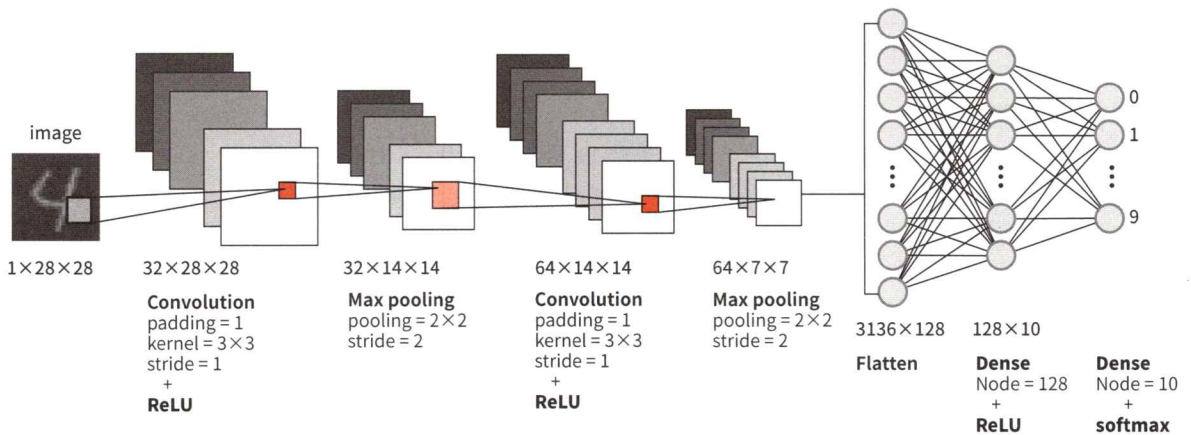
모델링

모델 클래스 생성하기

[그림 8-7]은 이번 차시 초반에 소개한 모델 구조입니다. 이번에는 [그림 8-7]의 구조대로 모델을 설계하고 학습하고 검증합니다.

[그림 8-7]의 모델 구조대로 다음과 같이 설계합니다.

- 입력: $1 \times 28 \times 28$ 크기의 이미지
- 합성곱층 1: 3×3 크기(kernel = 3×3)의 필터(커널) 32개로 특징 추출, 이때 입력 크기와 출력 크기가 같도록 패딩 설정(padding = 1)
- 맥스 풀링층 1: pool size = 2×2



[그림 8-7] MNIST의 CNN 모델 구조 예시

- 합성곱층 2: 64개 필터(커널)로 특징 추출(kernel = 3×3, padding = 1)
- 맥스 풀링층 2: pool size = 2×2
- Flatten
- 완전 연결층 1: node 128개
- 완전 연결층 2: node 10개(출력층)

CNN_model이라는 이름으로 모델 클래스를 생성합니다.

CODE

```
class CNN_model(nn.Module):
    def __init__(self):
        super(CNN_model, self).__init__()
        self.conv1 = nn.Sequential(nn.Conv2d(in_channels=1, out_channels=32, ①
                                              kernel_size=3, stride=1, padding=1),
                                   nn.ReLU(),
                                   nn.MaxPool2d(kernel_size=2, stride=2))
        self.conv2 = nn.Sequential(nn.Conv2d(in_channels=32, out_channels=64, ②
                                              kernel_size=3, stride=1, padding=1),
                                   nn.ReLU(),
                                   nn.MaxPool2d(kernel_size=2, stride=2))

        self.relu = nn.ReLU()
        self.flatten = nn.Flatten()
        self.fc1 = nn.Linear(64 * 7 * 7, 128) ③
        self.fc2 = nn.Linear(128, 10)

    def forward(self, x):
        x = self.conv1(x)
        x = self.conv2(x)
```

```

x = self.flatten(x)
x = self.relu(self.fc1(x))
x = self.fc2(x)
return x

```

- ① 합성곱층, ReLU 활성화 함수, 맥스 풀링층을 하나로 묶어 합성곱 블록을 선언합니다. 합성곱층, ReLU 활성화 함수, 맥스 풀링층이 CNN 모델을 구성하는 기본 단위이기 때문에 보통 가독성을 높이기 위해 이렇게 묶어서 선언합니다. 흑백 이미지 채널 1을 받아 32개 필터로 특징을 추출합니다.
- ② 두 번째 합성곱 블록입니다. 32개 특징 맵을 입력으로 받아 64개 필터로 특징을 추출합니다.
- ③ 합성곱 블록을 모두 거치고 펼친(Flatten) 다음, 완전 연결층으로 전달되는 특징 수는 $64 * 7 * 7$ 개입니다.

모델은 크게 두 그룹으로 이루어져 있습니다. 합성곱층 그룹과 완전 연결층 그룹입니다. 합성곱층 그룹에서 출력한 특징 맵을 펼쳐 1차원 형태로 만든 특징들은 완전 연결층으로 연결됩니다. 이때 완전 연결층으로 입력되는 특징 수를 계산할 수 있어야 합니다. 모델에서는 이 수가 $64 \times 7 \times 7 = 3,136$ 개입니다. 이 계산을 이해하는 것이 중요하기 때문에 이후에 나올 ‘층별 출력 크기 계산하기’ 소절에서 좀 더 자세히 알아보겠습니다. 지금은 설계된 모델을 선언하고 학습하고 예측/평가를 구현하는데 초점을 맞춥니다.

모델 학습 및 평가

모델을 선언하면서 디바이스를 할당하고 손실 함수와 옵티마이저를 준비합니다.

```

CODE
model2 = CNN_model().to(device)
loss_fn = nn.CrossEntropyLoss()
optimizer = Adam(model2.parameters(), lr=0.001)

```

모델을 학습하고 예측하고 평가하는 과정은 앞서 실습 1의 코드와 동일합니다. 실습 1의 코드에서 `train`, `evaluate` 함수의 매개변수를 `model`에서 `model2`로 변경하기만 하면 됩니다. 여기서는 모델 성능만 확인하겠습니다.

[그림 8-8]은 `classification_report` 함수를 사용해 모델을 평가한 결과입니다. 5일 차 다중 분류 모델의 정분류율은 0.95정도였고 앞서 구현한 단순 합성곱 모델은 0.9884였습니다. 이 수치도 상당히 높은 성능을 보여 주는 결과이지만, 합성곱층을 두 겹 쌓은 실습 2 모델은 거의 1에 가까운 정분류율(0.996)을 보여 줍니다.

	precision	recall	f1-score	support
0	1.0000	0.9981	0.9990	520
1	0.9982	1.0000	0.9991	564
2	0.9862	0.9960	0.9911	502
3	1.0000	0.9980	0.9990	510
4	1.0000	0.9938	0.9969	482
5	0.9977	0.9977	0.9977	436
6	0.9980	0.9980	0.9980	496
7	0.9922	0.9884	0.9903	516
8	0.9959	0.9959	0.9959	485
9	0.9918	0.9939	0.9928	489
accuracy			0.9960	5000
macro avg	0.9960	0.9960	0.9960	5000
weighted avg	0.9960	0.9960	0.9960	5000

[그림 8-8] 분류 모델 평가 요약 보고서 결과

모델의 성능이 기존 모델보다 훨씬 높아진 것을 확인할 수 있습니다.

한 가지 주의할 점이 있습니다. MNIST 데이터 세트는 딥러닝 입문용으로 많이 사용하지만, 조금은 '이상해 가까운' 데이터 세트라는 점입니다. MNIST 데이터 세트는 다음과 같은 이유로 높은 정확도가 비교적 쉽게 나옵니다.

- 이미지가 깨끗하고 정중앙에 위치해 있습니다. MNIST 손글씨 이미지들은 흰 배경에 또렷한 검정색으로 숫자가 쓰여 있고 한가운데에 잘 정렬되어 있습니다. 노이즈가 거의 없고 배경도 단순해 모델을 학습시키기 좋은 데이터입니다.
- 범주 간의 경계가 명확합니다. '0'과 '1', '7'과 '9'처럼 비슷한 숫자도 형태의 차이가 꽤 뚜렷한 편입니다. 따라서 모델이 혼동할 가능성이 낮아 분류가 쉬운 편입니다.
- 데이터 분포가 균형 잡혀 있습니다. 각 숫자(범주)의 개수가 비슷하게 분포되어 있어 학습할 때 특정 숫자에 편향되지 않습니다.

현실에서 만나는 데이터는 MNIST처럼 그렇게 '이상적'이지 않습니다. 예를 들어 손글씨 이미지는 흐리거나 비뚤어져 있을 수 있고 배경에는 다른 무늬나 그림자가 섞일 수도 있으며 숫자가 구석에 작게 쓰여 있거나 이미지 중심에서 벗어날 수도 있습니다. 또한 일부 범주(예: '3'이나 '5')는 많이 나오는 반면, 다른 범주는 상대적으로 매우 적을 수도 있습니다. 이런 상황에서는 모델의 정분류율이 많이 떨어집니다. 실무에서는 이와 같은 현실 세계의 다양한 특징들이 반영된 데이터를 모아 모델을 학습시킨다는 점에 주의해야 합니다.

층별 출력 크기 계산하기

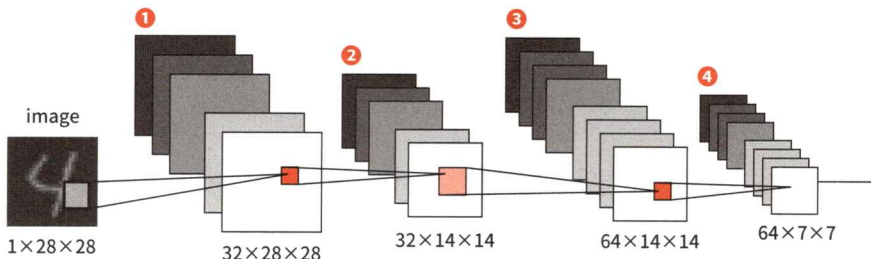
CNN을 구성할 때는 각각의 층을 거치며 이미지가 어떻게 변형되는지 이해하는 일이 중요합니다. 특히 합성곱층과 맥스 풀링층을 반복적으로 통과하고 완전 연결층(Dense Layer)으로 입력을 전달할 때는 정확한 출력 크기를 알고 있어야 합니다.

출력 크기를 확인하는 방법은 다음 두 가지입니다.

- 직접 계산해 보는 방법
- 코드를 실행하면서 print 문으로 구조(shape)를 확인하는 방법

출력 크기 직접 계산하기

[그림 8-9]는 CNN 모델에서 입력 이미지가 여러 층을 통과하며 크기가 어떻게 변하는지를 시각적으로 보여 줍니다.



[그림 8-9] 각각의 층별 출력 크기 변화

다음은 이 모델의 주요 구조를 구성하는 코드입니다.

CODE

```
nn.Conv2d(in_channels=1, out_channels=32, kernel_size=3, stride=1, padding=1), nn.ReLU(), ①
nn.MaxPool2d(kernel_size=2, stride=2), ②
nn.Conv2d(in_channels=32, out_channels=64, kernel_size=3, stride=1, padding=1), nn.ReLU(), ③
nn.MaxPool2d(kernel_size=2, stride=2), ④
```

- ① 첫 번째 합성곱층: 입력 이미지의 크기는 1x28x28입니다. nn.Conv2d 함수를 적용하면 출력은 32x28x28이 됩니다. 코드에서 padding=1이므로 너비와 높이는 그대로 유지됩니다. 출력이 32이므로 입력 채널 하나의 이미지에서 32개의 필터가 각각 연산을 수행한 결과, 총 32개의 특징 맵이 만들어집니다.
- ② 첫 번째 맥스 풀링층: 풀링은 이미지의 높이와 너비를 절반으로 줄이는 연산입니다. 입력 크기 32x28x28은 풀링 연산을 하면 32x14x14 크기가 됩니다. 채널 수(32)는 유지되는 대신 픽셀 수는 절반으로 줄어듭니다(28 // 2 = 14).

- ③ 두 번째 합성곱층: 다시 합성곱층을 통과하면 패딩 설정으로 인해 높이×너비 크기는 동일하지만, 채널 수는 64개로 증가합니다. 이제 출력 크기는 $64 \times 14 \times 14$ 입니다.
- ④ 두 번째 맥스 풀링층: 마지막으로 풀링 연산을 하면 $64 \times 7 \times 7$ 이 됩니다($14 // 2 = 7$).

이해를 돕고자 이 과정을 표로 정리했습니다. [표 8-1]을 보면 합성곱층을 통과할 때는 채널 수가 변하고, 맥스 풀링층을 통과할 때는 세로와 가로 픽셀 수가 변합니다.

[표 8-1] 합성곱층과 맥스 풀링층을 통과할 때의 데이터 크기 변화

층	입력 크기	출력 크기
① 합성곱층 1	[1, 28, 28]	[32, 28, 28]
② 맥스 풀링층 2	[32, 28, 28]	[32, 14, 14]
③ 합성곱층 2	[32, 14, 14]	[64, 14, 14]
④ 맥스 풀링층 2	[64, 14, 14]	[64, 7, 7]

[표 8-1]에서 정리했듯이 층과 연산을 거치면서 데이터의 크기가 어떻게 변하는지 이해했다면 다음 문제를 한 번 풀어 봅시다. 만약 kernel size, padding, stride 옵션이 동일할 때, 합성곱과 맥스 풀링 연산을 한 번 더 추가한다면 크기는 어떻게 변할까요? 이때 합성곱층의 채널 수는 128로 증가하도록 구성합니다. 다음 [표 8-2]의 물음표(?)에 들어갈 숫자를 계산해 보세요.

[표 8-2] 합성곱층과 맥스 풀링층 추가

합성곱층 3	[64, 7, 7]	[128, ?, ?]
맥스 풀링층 3	[?, ?, ?]	[?, ?, ?]

풀이를 보기 전에 먼저 손으로 직접 계산해 보기 바랍니다.

[표 8-2]를 보면 합성곱층은 64개의 입력 채널이 128개의 입력 채널로 늘어났습니다. 이때 높이×너비의 크기는 어떻게 될까요? 높이×너비의 크기는 동일합니다. 그러므로 합성곱층의 출력 크기는 $128 \times 7 \times 7$ 입니다.

다음으로 맥스 풀링층의 입력 크기는 합성곱층의 출력 크기($128 \times 7 \times 7$)와 동일합니다. 맥스 풀링층의 출력 크기는 입력 크기에서 높이×너비의 절반이 됩니다. $7 // 2 = 3$ 이므로 $128 \times 3 \times 3$ 이 됩니다. 퀴즈를 맞췄다면 여러분은 이 계산법을 잘 이해한 겁니다.

print 문으로 크기 출력하기

지금까지 층마다 출력되는 데이터의 크기를 직접 계산했습니다. 하지만 모델 구조가 복잡해지면 모든 층의 출력 크기를 일일이 계산하는 일은 어렵고 실수가 생길 수도 있습니다.

순전파(forward) 메서드에 print 문을 추가하면 각각의 층에서 출력되는 텐서의 구조(shape)를 직접 확인할 수 있어 효과적입니다.

이번에는 CNN_model2라는 이름으로 간단한 CNN 모델을 구성하고 각각의 층마다 출력 크기를 출력하도록 만듭니다. '8일차_이미지모델링실습1.ipynb' 파일에서 이어 작성하면 됩니다.

CODE

```
class CNN_model2(nn.Module):
    def __init__(self):
        super(CNN_model2, self).__init__()
        self.conv1 = nn.Conv2d(in_channels=1, out_channels=32, kernel_size=3,
                                stride=1, padding=1)
        self.pool1 = nn.MaxPool2d(kernel_size=2, stride=2)
        self.conv2 = nn.Conv2d(in_channels=32, out_channels=64, kernel_size=3,
                                stride=1, padding=1)
        self.pool2 = nn.MaxPool2d(kernel_size=2, stride=2)
        self.relu = nn.ReLU()
    def forward(self, x):
        print(f"입력 크기: {x.shape}")           ①
        x = self.relu(self.conv1(x))
        print(f"Conv1 출력 크기: {x.shape}")     ②
        x = self.pool1(x)
        print(f"MaxPool1 출력 크기: {x.shape}")  ③
        x = self.relu(self.conv2(x))
        print(f"Conv2 출력 크기: {x.shape}")     ④
        x = self.pool2(x)
        print(f"MaxPool2 출력 크기: {x.shape}")  ⑤
        return x
```

- ① 입력 데이터를 받았을 때의 초기 출력 크기를 확인합니다.
- ② 첫 번째 합성곱층의 출력 크기를 확인합니다.
- ③ 첫 번째 맥스 풀링층에서 연산을 수행하고 출력 크기를 확인합니다.
- ④ 두 번째 합성곱층의 출력 크기를 확인합니다.
- ⑤ 두 번째 맥스 풀링층에서 연산을 수행하고 최종 출력 크기를 확인합니다.

이 모델에 실제 데이터를 보내면서 출력 크기를 확인하겠습니다.

다음은 모델을 선언하고 테스트 데이터의 일부를 넣어 출력을 확인하는 코드입니다.

CODE

```
model2 = CNN_model2() ①
input = x_test[0:10] ②
output = model2(input) ③
```

OUTPUT

```
입력 크기: torch.Size([10, 1, 28, 28])
Conv1 출력 크기: torch.Size([10, 32, 28, 28])
MaxPool1 출력 크기: torch.Size([10, 32, 14, 14])
Conv2 출력 크기: torch.Size([10, 64, 14, 14])
MaxPool2 출력 크기: torch.Size([10, 64, 7, 7])
```

- ① CNN_model2 클래스를 이용해 모델을 생성합니다.
- ② 테스트 데이터 이미지에서 앞쪽 10개의 이미지를 선택해 입력 데이터로 사용합니다.
- ③ forward 메서드를 호출하면 각각의 층에 설정한 print 문이 실행됩니다.

실행 결과를 보면 입력 데이터는 10개의 이미지로 구성된 배치(Batch)이며 각 이미지의 크기는 $1 \times 28 \times 28$ 입니다. 모델을 통과하면서 출력 크기가 점점 변하고 있음을 확인할 수 있습니다.

- 채널 수 변화: 합성곱층을 지나면서 채널 수는 $1 \rightarrow 32 \rightarrow 64$ 로 증가합니다. 이는 각각의 층에서 더 다양한 특징을 추출하기 위해 필터 수를 늘리기 때문입니다.
- 가로×세로 크기 변화: 맥스 풀링 연산을 거치면서 $28 \times 28 \rightarrow 14 \times 14 \rightarrow 7 \times 7$ 로 점 점 줄어듭니다. 풀링 연산은 공간의 크기와 계산량을 줄이는 효과가 있습니다.

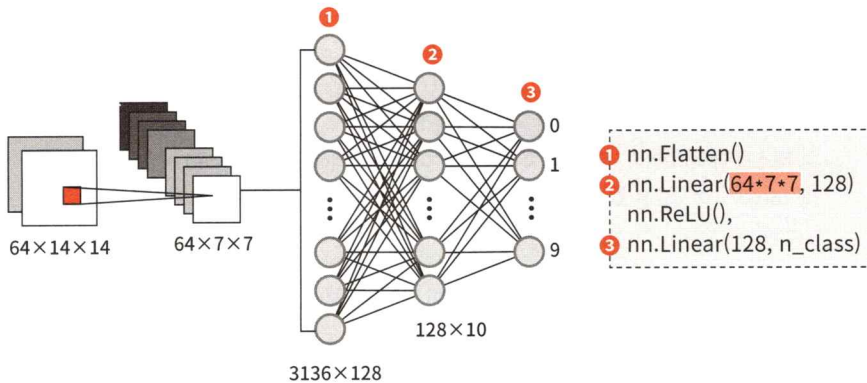
실무에서도 print 문을 사용해 각 층의 출력을 직접 확인하는 일이 매우 중요합니다. 특히 마지막 맥스 풀링 연산 이후, 데이터를 펼쳐(Flatten) 완전 연결층(nn.Linear)으로 데이터를 넘기기 전에는 텐서의 크기를 반드시 확인해야 합니다. 그래야 올바른 차원으로 연결할 수 있습니다. 예를 들어 마지막 맥스 풀링층의 출력 크기가 $[n, 64, 7, 7]$ 이라면 nn.Linear 층의 입력 차원은 $64 \times 7 \times 7 = 3,136$ 이 되어야 합니다. 여기서 n은 배치 크기(Batch Size)이므로 층의 노드 수와는 관계가 없습니다. 즉, 완전 연결층으로 전달되는 한 개의 데이터는 64채널의 7×7 크기를 가진다는 뜻입니다.

펼치고(Flatten) 완전 연결층으로 연결하기

이제 CNN이 추출한 특징을 완전 연결층(Dense Layer)으로 전달해야 하는데, 완전 연결층은 1차원 데이터를 입력으로 받습니다. 그러나 CNN이 추출한 특징 맵(Feat-

ture Map)은 2차원 또는 3차원 형태의 데이터입니다. 따라서 2차원(또는 3차원) 형태의 특징 맵을 1차원으로 펼쳐 주는(Flatten) 과정이 필요합니다.

[그림 8-10]은 특징 맵을 펼치고 완전 연결층(Dense Layer)으로 연결하는 과정을 다이어그램과 코드 구현으로 보여 주고 있습니다.



[그림 8-10] Flatten + 완전 연결층

[그림 8-10]의 과정을 단계별로 설명하면 다음과 같습니다.

- 1 값 펼치기(Flatten): Flatten 함수는 여러 차원의 데이터를 옆으로(한 줄로) 펼쳐 줍니다. CNN을 통과한 최종 출력 데이터는 [채널, 높이, 너비] 형태입니다. 예컨대 이전 합성곱층의 최종 출력이 [64, 7, 7]이라면 이를 1차원 벡터로 변환(Flatten)해야 합니다. Flatten 함수를 사용하면 결과의 크기는 $64 \times 7 \times 7 = 3,136$ 입니다.
- 2 `nn.Linear`로 연결: 입력 데이터의 크기는 이전 층(CNN + MaxPool)의 출력을 펼친(Flatten) 크기입니다. 앞서 계산한 3,136입니다.
- 3 출력층: 완전 연결층(`nn.Linear`)으로 연결되며 분류 모델의 출력층 역할을 합니다.

값을 펼친(Flatten) 이후에는 기존의 회귀나 분류 모델과 똑같이 은닉층과 출력층을 연결해 모델 구조를 설계하면 됩니다.

MNIST 모델에 대한 수동 입력 테스트

지금까지는 모델을 만들고 검증 평가만 해봤는데, 이번에는 방금 만든 모델을 사용해 보겠습니다. 여러분이 직접 쓴 손글씨를 모델에 입력해서 0~9까지의 숫자를

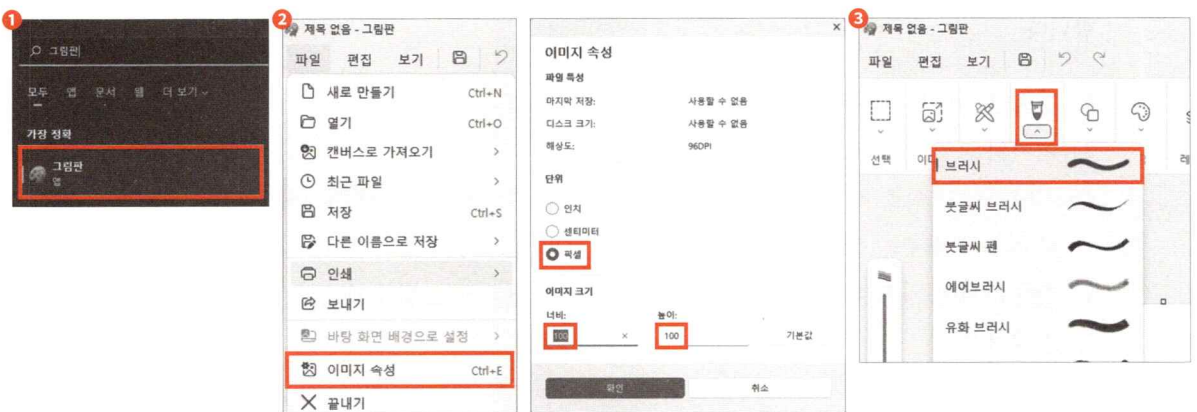
잘 맞추는지 확인하겠습니다. 실습은 전과 동일하게 '8일차_이미지모델링실습1.ipynb' 파일에서 이어 진행합니다.

이번 실습은 Windows의 '그림판' 프로그램을 기반으로 설명하지만, 그림을 그릴 수 있는 다른 프로그램을 이용해도 무방합니다. 다음과 같은 순서로 진행합니다.

1. 그림판에서 마우스로 직접 쓴 손글씨를 이미지 파일로 저장합니다.
2. 학습할 때의 데이터 구조에 맞게 이미지 데이터를 전처리합니다.
3. 모델에 입력하고 예측 결과를 얻습니다.

손글씨로 그리기

여러분이 사용하는 PC의 운영체제는 다양하겠지만, 여기서는 Windows의 '그림판' 프로그램을 기준으로 설명합니다.



[그림 8-11] 그림판 준비

- ❶ 그림판을 실행합니다. 키보드 왼쪽 아래에 있는 윈도우 버튼을 누른 다음, 하단 검색 폼에서 그림판이라고 입력하면 '그림판' 프로그램이 검색됩니다. 그림판을 클릭합니다.
- ❷ 빈 그림판이 하나 열립니다. [파일] - [속성] 메뉴를 클릭하면 [이미지 속성] 대화 상자가 나옵니다. 이미지의 크기를 100×100 정도로 조정하고 [확인] 버튼을 클릭합니다.
- ❸ 그림판 상단의 아이콘 패널에서 펜 타입을 브러시로 선택합니다.

그림판에서 펜 브러시의 두께를 10px 정도로 키운 다음, 0~9 사이의 숫자 중 하나를 마우스로 그립니다. 필자는 숫자 2를 그려 보았습니다.



[그림 8-12] 손글씨 숫자 2

이 그림 파일을 저장합니다. 저장할 때 확장자는 jpg나 png 등을 선택합니다.

이미지 전처리

방금 손으로 쓴 숫자 이미지를 모델에 입력하려면 몇 가지 전처리 절차를 거쳐야 합니다.

1. 필요한 라이브러리 로딩
2. 이미지 파일 업로드
3. 이미지를 변수로 저장하고 이미지 크기 조정: 학습할 때 사용한 이미지의 크기와 컬러에 맞추는 작업
4. 학습할 때 배치 단위로 데이터를 입력하므로 배치 단위 구조 맞추기
5. 스케일링
6. 텐서 변환

전처리 단계를 하나씩 적어보니 해야 할 일이 꽤 많아 보입니다. 하지만 코드는 간단합니다.

먼저 필요한 라이브러리를 로딩합니다.

CODE

```
import cv2 ①  
from google.colab.patches import cv2_imshow ②
```

① OpenCV 라이브러리: 영상 및 이미지를 처리하는 라이브러리입니다.

② 구글 코랩에서 cv2를 기반으로 이미지를 화면에 보여 주기 위한 함수를 불러옵니다.

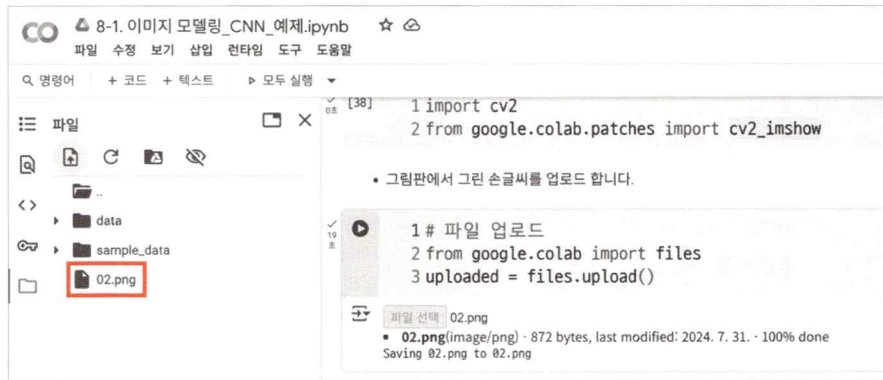
이미지 파일을 업로드합니다. `files.upload` 코드는 코랩에서 파일을 업로드하는 함수입니다.

CODE

```
from google.colab import files
uploaded = files.upload()
```

코드를 실행하면 <파일 선택> 버튼이 나옵니다. 버튼을 클릭해 저장할 이미지를 선택합니다.

이미지를 업로드하면 다음과 같이 코랩 창의 '파일' 탭에서 이미지 파일이 보입니다. 필자가 업로드한 파일은 '02.png'입니다.



[그림 8-13] 코랩에 이미지 업로드하기

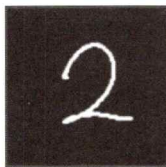
다음으로 이미지를 변수로 저장하고 크기도 조절합니다.

CODE

```
img = cv2.imread('02.png', cv2.IMREAD_GRAYSCALE) ①
img = cv2.resize(255-img, (28, 28))                ②
print(img.shape) ③
cv2_imshow(img) ④
```

OUTPUT

```
(28, 28)
```



- ① 업로드한 이미지('02.png')를 변수 `img`에 저장합니다. 이때 `cv2.IMREAD_GRAYSCALE` 옵션을 사용해 흑백으로 변환합니다. MNIST 데이터를 학습할 때 이미지 채널이 1(흑백)이므로 이 데이터 형식에 맞춥니다. 원본 이미지가 이미 흑백이므로 꼭 필요한 작업은 아니지만, 데이터를 학습시킬 때 데이터의 구조와 동일하게 맞추어야 한다는 점을 기억하면 됩니다.

- ② 손글씨 이미지는 흰 배경에 검은 글씨로 되어 있지만, MNIST 데이터 세트는 검은 배경에 흰 글씨입니다. 따라서 `255-img` 연산으로 색상을 반전시킵니다(255는 픽셀의 최댓값으로 흑백 이미지에서 밝기를 0~255로 표현합니다). 그리고 (28, 28) 크기로 조정해 MNIST 모델이 학습한 입력 크기와 동일하게 맞춥니다.
- ③ 이미지 데이터가 모델에 들어가기 전에 원하는 크기로 잘 변환되었는지 확인하는 단계입니다. 출력 결과는 (28, 28)인데, 높이 28, 너비 28 크기의 2차원 배열임을 의미합니다.
- ④ 전처리가 잘 되었는지 눈으로 직접 확인하기 위해 이미지를 화면에 출력합니다.

나머지 전처리 과정은 배치로 변환, 스케일링, 텐서 변환 작업입니다.

CODE

```
new_data = img.reshape(1,1,28,28) ①
new_data = new_data / 255 ②
new_data = torch.tensor(new_data, dtype=torch.float32) ③
```

- ① CNN 모델은 이미지를 4차원 텐서 형태로 받습니다. 차원 구성은 [배치 크기, 채널 수, 높이, 너비]입니다. 따라서 (28, 28) 형태의 2차원 데이터를 (1, 1, 28, 28)로 바꿔 모델이 처리할 수 있도록 합니다.
- ② 스케일링을 수행합니다.
- ③ 텐서로 변환합니다.

다음으로 텐서 데이터 세트 형태의 입력 데이터를 모델에 넣으면 예측 결과를 얻을 수 있습니다.

CODE

```
model.eval() # 모델을 평가 모드로 전환
with torch.no_grad(): # 그래디언트 계산 비활성화(메모리 사용을 줄이고 속도 향상)
    new_data = new_data.to(device) # 입력 데이터를 모델과 동일한 디바이스로 이동
    pred = model(new_data) # 모델에 입력 데이터를 넣어 예측 수행
    pred = nn.functional.softmax(pred, dim=1) # 범주별 확률로 변환(소프트맥스 적용)
    pred = np.argmax(pred.cpu().numpy(), axis = 1) # 가장 확률 높은 범주의 인덱스 추출

print(pred)
```

OUTPUT

```
[2]
```

코드는 이미 다룬 내용이므로 코드 옆의 주석으로 간략히 설명했습니다. 실행 결과를 보면 2라고 적은 이미지를 [2]로 잘 예측하고 있습니다. 여러분이 쓴 손글씨 숫자 이미지도 잘 예측했나요?

다음은 컬러 이미지를 분류하는 모델링 실습도 해보겠습니다.

실습 3: 컬러 이미지 분류 모델링

지금까지 다룬 MNIST 데이터는 흑백 이미지입니다. 이번 실습은 컬러 이미지 데이터 세트로 유명한 CIFAR-10을 이용해 분류 모델링을 수행하는 실습입니다. 새로운 데이터로 수행하는 실습이므로 새 코드 파일을 열어서 코드를 작성하겠습니다. 새 코드 파일의 이름은 '8일차_이미지모델링실습2.ipynb'로 합니다. 이번 파일도 GPU를 사용하도록 런타임을 설정하고 시작합니다.

환경 준비부터 데이터 전처리까지

라이브러리 로딩, 디바이스 준비, 사용자 정의 함수 생성은 앞에서 생성한 '8일차_이미지모델링실습1.ipynb' 파일 코드에서 그대로 가져오면 됩니다. 이어서 CIFAR-10 데이터를 불러와 소개하는 과정부터 다시 시작하겠습니다.

CIFAR-10 데이터 불러오기

이미지 분류 실습에서 널리 사용되는 데이터 세트 중 하나가 CIFAR-10입니다. 이 데이터 세트는 컴퓨터 비전 분야의 '기본 교과서'로 불릴 만큼 많이 활용되는 데이터이기 때문에 딥러닝 모델을 실험하고 비교하는 표준 데이터 세트로 자리 잡았습니다.

우선 다음 코드로 데이터를 불러옵니다.

CODE

```
from torchvision import datasets
from torchvision.transforms import ToTensor

train_dataset = datasets.CIFAR10(root="data", train=True, download=True,
                                  transform=ToTensor()) ①
test_dataset = datasets.CIFAR10(root="data", train=False, download=True,
                                  transform=ToTensor())
```

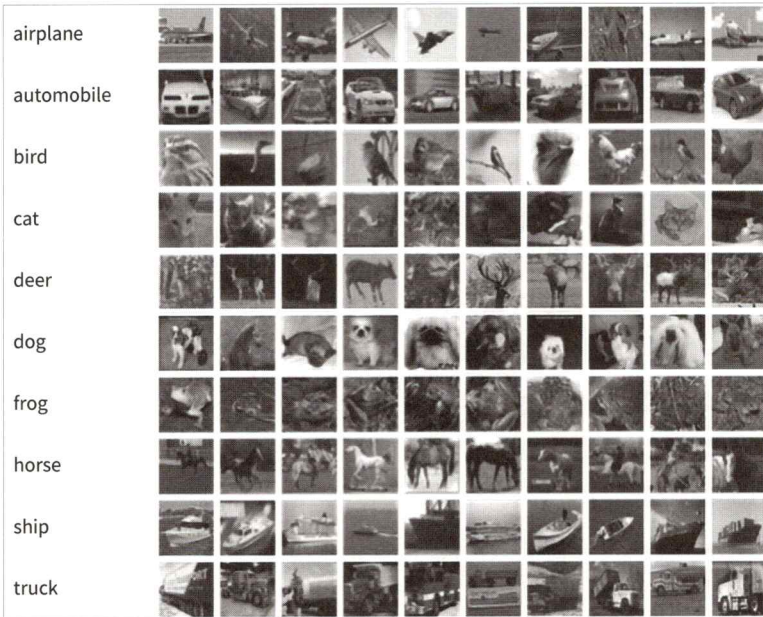
OUTPUT

100%|██████████| 170M/170M [00:16<00:00, 10.5MB/s]

① torchvision의 datasets 모듈에 있는 CIFAR10 함수로 데이터를 다운로드합니다. 문법은 MNIST 다운로드 문법과 동일합니다.

CIFAR-10은 총 60,000장의 컬러 이미지로 구성되어 있고 10개의 서로 다른 범주(종류)를 포함합니다. 10개의 범주에는 비행기(Airplane), 자동차(Automobile), 새(Bird), 고양이(Cat), 사슴(Deer), 개(Dog), 개구리(Frog), 말(Horse), 배(Ship), 트럭

(Truck)이 포함됩니다. 즉, 사람과 동물, 탈것 등 다양한 사물이 포함되어 있어 모델이 여러 유형의 패턴을 학습하기에 적합합니다.



[그림 8-14] CIFAR-10(10가지 사물에 대한 컬러 이미지 데이터 세트)

CIFAR-10은 50,000장의 학습용 이미지와 10,000장의 테스트용 이미지로 구성되어 있습니다. 이미지는 각각 32×32 픽셀 크기이고 3개의 채널(RGB)로 구성된 컬러 이미지입니다.

다음 코드를 실행합니다.

CODE

```
train_dataset.data.shape
```

OUTPUT

```
(50000, 32, 32, 3)
```

학습 데이터 세트에서 입력 이미지의 구조(shape)를 확인하면 세로 32픽셀, 가로 32픽셀의 컬러 이미지($32 \times 32 \times 3$)가 50,000개를 알 수 있습니다.

범주의 종류도 확인합니다.

CODE

```
classes = train_dataset.classes
classes
```

OUTPUT

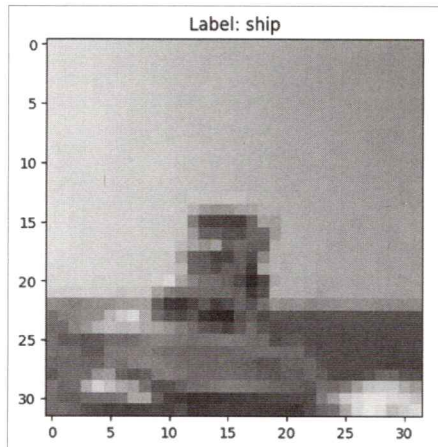
```
['airplane',  
 'automobile',  
 'bird',  
 'cat',  
 'deer',  
 'dog',  
 'frog',  
 'horse',  
 'ship',  
 'truck']
```

train_set의 classes 속성을 이용해 10개의 목푫값이 어떤 범주인지 확인할 수 있습니다.

이번에는 이미지 한 개를 출력합니다.

CODE

```
n = 100  
image, label = train_dataset.data[n], train_dataset.targets[n]  
plt.imshow(image)  
plt.title(f"Label: {train_dataset.classes[label]}")  
plt.show()
```

OUTPUT

학습 데이터 세트에 있는 인덱스 100의 그림입니다. 목푫값은 'ship'입니다. 해상도가 낮아 배(ship)가 선명하게 보이지는 않습니다. 이런 형태의 컬러 이미지들로 CNN 모델을 학습시킵니다.

데이터 전처리

데이터 전처리 단계에서는 학습 데이터를 위한 데이터 로더와 검증 및 테스트를 위한 텐서 데이터 세트를 준비합니다. MNIST 데이터와는 조금 다른 부분이 있으니 주의하면서 다음과 같이 코드를 작성합니다.

CODE

```
# 학습 데이터의 데이터 로더
train_loader = DataLoader(train_dataset, batch_size = 64)

# 검증, 테스트 데이터를 위한 텐서 데이터 세트 준비
## val, test 분할
x_val, x_test = test_dataset.data[:5000], test_dataset.data[5000:]
y_val, y_test = test_dataset.targets[:5000], test_dataset.targets[5000:]

## 스케일링
x_val = x_val / 255
x_test = x_test / 255

## 차원 맞추기 - 기존 차원: [데이터 건수, 세로 픽셀, 가로 픽셀, 채널(컬러)]
print('기존: ', x_val.shape, x_test.shape)

## 차원 위치 조정: [데이터 건수, 채널(컬러), 세로 픽셀, 가로 픽셀]
x_val = x_val.transpose((0, 3, 1, 2)) ①
x_test = x_test.transpose((0, 3, 1, 2))
print('조정: ', x_val.shape, x_test.shape)

## 텐서로 변환
x_val = torch.tensor(x_val, dtype=torch.float32) ②
x_test = torch.tensor(x_test, dtype=torch.float32)
y_val = torch.tensor(y_val, dtype=torch.long)
y_test = torch.tensor(y_test, dtype=torch.long)
```

OUTPUT

```
기존: (5000, 32, 32, 3) (5000, 32, 32, 3)
조정: (5000, 3, 32, 32) (5000, 3, 32, 32)
```

- ① 텐서 데이터에서 사용된 `transpose` 메서드는 차원의 순서를 변경합니다. CIFAR10 데이터 세트는 [데이터 건수(0), 세로 픽셀(1), 가로 픽셀(2), 채널(3)]으로 저장되어 있습니다. 하지만 이미지 모델링을 위해서는 [데이터 건수(0), 채널(3), 세로 픽셀(1), 가로 픽셀(2)] 형식, 즉 채널이 두 번째에 오도록 순서를 변경해야 합니다. `transpose((0, 3, 1, 2))`를 이용해 기존 순서 (0, 1, 2, 3)을 (0, 3, 1, 2)로 바꿉니다.
- ② CIFAR10의 원본 데이터는 넘파이 배열과 리스트로 되어 있습니다. 따라서 텐서로 변환합니다.

데이터의 준비를 완료했습니다. 이제 모델링 단계로 넘어갑니다.

모델링

모델 설계

모델을 선언하고 학습 및 검증 평가를 수행합니다. 모델 구조를 다음과 같이 작성합니다.

CODE

```
class CNN_model3(nn.Module):
    def __init__(self):
        super(CNN_model3, self).__init__()
        self.conv1 = nn.Sequential(nn.Conv2d(in_channels=3, out_channels=32, ①
                                              kernel_size=3, stride=1, padding=1),
                                   nn.ReLU(),
                                   nn.MaxPool2d(kernel_size=2, stride=2))
        self.conv2 = nn.Sequential(nn.Conv2d(in_channels=32, out_channels=64,
                                              kernel_size=3, stride=1, padding=1),
                                   nn.ReLU(),
                                   nn.MaxPool2d(kernel_size=2, stride=2))

        self.relu = nn.ReLU()
        self.flatten = nn.Flatten()
        self.fc1 = nn.Linear(64 * 8 * 8, 128) ②
        self.fc2 = nn.Linear(128, 10)

    def forward(self, x):
        x = self.conv1(x)
        x = self.conv2(x)
        x = self.flatten(x)
        x = self.relu(self.fc1(x))
        x = self.fc2(x)
        return x
```

① `nn.Conv2d(in_channels=3, out_channels=32,...)`: 바로 이 부분이 MNIST 모델과 차이가 나는 지점입니다. MNIST 데이터는 흑백(채널=1) 이미지입니다. 따라서 `in_channels=1` 이었는데, CIFAR10 데이터는 컬러 이미지이므로 `in_channels=3`입니다.

② 두 번째 합성곱층+맥스 풀링층을 통과한 다음, 펼쳐서 완전 연결층에 연결할 때 전달할 입력값의 차원 수를 계산해서 수정해야 합니다. 직전 합성곱층의 출력 채널이 64였고 입력된 이미지의 가로와 세로 픽셀 수가 32이므로 $64 \times 8 \times 8$ 입니다($32 // 2 // 2 = 8$).

이 과정을 정리하면 [표 8-3]과 같습니다. 합성곱층을 통과할 때는 채널의 크기만 변하고 맥스 풀링층을 통과하면 세로와 가로 픽셀 수가 줄어듭니다.

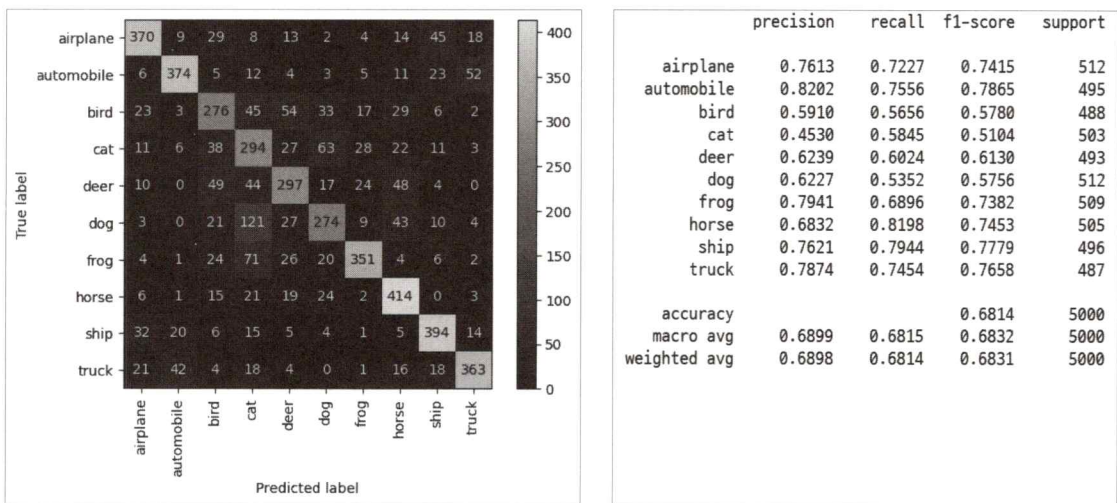
[표 8-3] 컬러 이미지를 입력받아 합성곱층+맥스 풀링층 블록을 통과할 때의 크기 변화

층	입력 크기	출력 크기
합성곱층 1	[3, 32, 32]	[32, 32, 32]
맥스 풀링층 2	[32, 32, 32]	[32, 16, 16]
합성곱층 2	[32, 16, 16]	[64, 16, 16]
맥스 풀링층 2	[64, 16, 16]	[64, 8, 8]

CIFAR-10처럼 컬러 이미지를 다룰 때는 입력 채널 수를 3으로 설정해야 합니다. 첫 번째 합성곱층의 `in_channels` 값만 조정하고 나머지 모델 구조는 MNIST 모델과 동일하게 설계하면 됩니다.

모델 선언, 학습 및 모델 평가

모델을 선언하고 학습하고 예측 및 평가하는 부분은 MNIST 모델의 코드와 동일하므로 여기에서도 성능 평가 결과만 확인합니다. 앞의 실습 과정을 참고해 여러분들이 직접 구현하고 확인하기 바랍니다. 참고로 완성된 코드는 깃허브 링크 https://github.com/DA4BAM/pytorch_deeplearning_basic에서 확인할 수 있습니다.



[그림 8-15] CIFAR10 모델 성능 평가 결과

전체 성능을 나타내는 정분류율은 0.6814로 그렇게 좋은 성능은 아닙니다. 범주별 성능을 비교해 보면 F1 score가 가장 높은 범주는 자동차(automobile)로서 0.7855

이고, 가장 낮은 범주는 고양이(cat)로서 0.5104입니다.

컬러 이미지의 합성곱 연산

마지막으로 컬러 이미지의 합성곱 연산에 대한 간단한 설명을 덧붙이면서 이번 차시를 마무리하겠습니다.

1채널 vs. 3채널 비교

MNIST의 흑백 이미지는 채널 수가 1이므로 합성곱 연산을 설명할 때는 주로 2차원 이미지(높이×너비)에 2차원 필터(예: 3×3)를 적용하는 방식으로 설명했습니다. 하지만 실제로는 이 필터에도 ‘채널’이라는 세 번째 축이 존재합니다. 흑백 이미지는 채널이 1이므로 필터도 실제로는 $1 \times 3 \times 3$ 형태입니다. 여기서 1은 채널 축이고 3×3 은 공간 영역입니다.

그런데 CIFAR-10과 같은 컬러 이미지는 RGB 3채널로 구성되어 있습니다. 즉, 이미지의 크기가 $3 \times 32 \times 32$ (채널×높이×너비)인 셈입니다. 이때 사용하는 필터는 단순한 2차원 형태가 아니기 때문에 3개의 채널을 동시에 처리할 수 있도록 필터 역시 3차원으로 구성합니다. 예를 들어 필터의 공간 크기가 3×3 이라면 컬러 이미지에 적용되는 필터는 $3 \times 3 \times 3$ 형태가 되어야 합니다. 여기서 3은 채널 수, 그리고 3×3 은 공간 크기입니다.

정리하면 입력 이미지가 가지고 있는 채널 수와 같은 수의 채널을 필터도 가져야 합성곱 연산을 할 수 있습니다. 따라서 컬러 이미지는 필터가 자동으로 3개의 채널을 갖도록 확장되고 이미지의 모든 채널(R, G, B)을 동시에 고려하면서 하나의 출력값을 계산합니다.

[표 8-4] 1채널 vs 3채널 비교

구분	흑백 이미지	컬러 이미지
채널 수	1	3(R, G, B)
이미지 크기	(H, W)	(C, H, W) = (3, H, W)
필터 크기	(1, k, k)	(3, k, k)

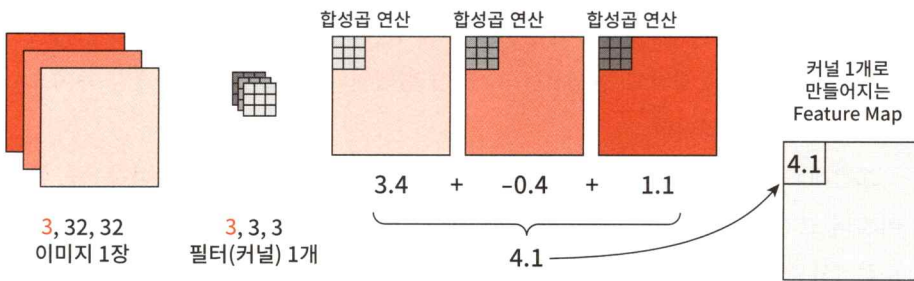
입력 이미지가 컬러(채널 3)이고 첫 번째 층의 필터 크기를 3(kernel_size = 3)으로 지정한다면 필터의 크기는 [3(채널 수)×필터 크기]가 됩니다. 즉 ($3 \times 3 \times 3$)이 됩니다.

다. 제일 앞에 있는 숫자인 3이 입력 이미지의 채널 수입니다. 코드를 작성할 때는 필터의 높이와 너비 크기만 지정하지만, 여기에 입력 채널의 수를 곱합니다. 많은 분들이 이 부분을 어려워합니다.

컬러 이미지 합성곱 연산

그렇다면 컬러 이미지를 입력할 때 합성곱층의 연산은 어떻게 수행할까요? [그림 8-16]으로 자세히 설명합니다.

```
nn.Conv2d(in_channels=3, out_channels=8, kernel_size=3, stride=1, padding=1)
```



[그림 8-16] 컬러 이미지를 입력할 때 합성곱 연산 예제

[그림 8-16]의 맨 왼쪽 그림은 컬러 이미지 한 개입니다. 크기는 $3 \times 32 \times 32$ 로 맨 앞의 3은 채널 수(Red, Green, Blue)입니다. 즉, 이 이미지는 32×32 픽셀 이미지가 R, G, B 3개의 채널로 나뉘어 저장됩니다. 이와 같은 3차원 구조의 이미지를 CNN에 입력합니다.

[그림 8-16]의 왼쪽에서 두 번째 작은 정육면체 이미지는 필터(kernel)입니다. 이 필터는 3개의 3×3 매트릭스로 구성되어 있습니다. 그리고 각각의 3×3 필터는 R, G, B 채널에 각각 하나씩 적용됩니다. 즉, $3 \times 3 \times 3$ 필터는 R, G, B 세 개의 채널에서 같은 위치에 있는 3×3 영역에 각각 적용되어 합성곱 연산을 수행한 다음, 그 결과값들을 모두 더해 최종 하나의 출력값을 만듭니다.

이제 필터는 채널마다 같은 위치에 있는 3×3 영역에서 합성곱 연산을 수행합니다.

- Red 채널에서 3×3 필터 적용 → 예: 값 = 3.4
- Green 채널에서 3×3 필터 적용 → 예: 값 = -0.4
- Blue 채널에서 3×3 필터 적용 → 예: 값 = 1.1

각각의 채널에서 계산된 결과값을 모두 더합니다.

$$3.4 - 0.4 + 1.1 = 4.1$$

각각의 채널에서 추출한 계산 결과를 모두 더하면 하나의 위치에 대한 출력값 4.1이 생성됩니다. 이렇듯 동일한 위치에 있는 3개의 채널 필터로 합성곱한 결과를 합쳐 하나의 숫자(특징)를 만듭니다. 이 과정을 이미지 전체에서 반복하면 원래 $3 \times 32 \times 32$ 크기의 입력 이미지는 1개의 2차원 특징 맵(Feature Map)으로 변환됩니다.

컬러 이미지는 세 개의 채널(R, G, B)을 포함하므로 CNN은 이들을 동시에 고려해 색상 정보를 반영한 특징을 추출합니다. 따라서 흑백 이미지와 마찬가지로 2차원으로 필터가 이동하지만, 필터 자체는 이미지의 채널 수만큼 깊어져 3차원 형태로 작동합니다. 이런 구조 덕분에 CNN은 컬러 이미지의 풍부한 색상 정보를 반영한 고유의 특징 맵을 만들 수 있습니다.

정리

이번 차시에서는 이미지 데이터를 효과적으로 다루는 대표적인 딥러닝 모델인 합성곱 신경망(CNN)의 기본 개념과 구조, 그리고 실제 이미지 처리 과정을 학습했습니다. 먼저 CNN은 사람이 이미지를 볼 때처럼 전체를 한꺼번에 인식하지 않고 일부 영역(지역적 특징)을 바탕으로 중요한 정보를 추출하는 식으로 동작합니다. 예를 들어 고양이를 인식할 때 귀, 눈, 코와 같은 부위를 보고 판단하듯이 CNN도 필터로 이미지의 작은 영역을 순차적으로 분석합니다. 필터 과정을 거쳐 이미지는 수치로 표현 가능한 특징 맵(Feature Map)으로 변환되는데, 학습이 진행될수록 더 복잡한 패턴까지 인식할 수 있습니다.

이어진 내용에서는 CNN의 구성 요소인 합성곱층(Conv2D), ReLU 활성화 함수, 풀링(MaxPooling2D) 연산의 구조와 역할을 자세히 설명했습니다. 특히 합성곱 연산에서는 작은 필터(예: 3×3)가 이미지의 일부분을 스캔하면서 각각의 영역에서 곱셈 및 덧셈을 수행해 새로운 출력값을 생성합니다. 이때 흑백 이미지의 경우 필터는 2차원(1채널)이지만, 컬러 이미지(RGB)는 3채널로 구성되어 있어 필터도 $3 \times 3 \times 3$ 의 3차원 형태로 확장됩니다. 각각의 채널(R, G, B)에서 필터를 별도로 적용한 다음, 그 결과를 합산하여 하나의 출력값을 생성합니다. 이 과정을 이미지 전체에 반복하

면 입력 이미지가 CNN을 통과하면서 여러 개의 특징 맵으로 변환됩니다. 이때 이미지는 모델이 학습에 활용할 수 있는 유용한 표현으로 바뀝니다.

마지막으로 CNN의 출력은 3차원 텐서 형태이므로 완전 연결층(Dense Layer)에 전달하기 전에 Flatten 연산을 통해 1차원 벡터로 변환해야 합니다. 변환된 벡터는 `nn.Linear`를 통해 기존 딥러닝 모델 구조와 동일한 회귀나 분류 모델로 연결됩니다.

지금까지 배운 내용은 이미지 모델링의 기본입니다. 많은 이미지 모델이 CNN을 기반으로 발전했습니다. 추가로 더 공부하기 원하는 분들은 Object Detection 모델인 YOLO(You Only Look Once)를 공부하길 권장합니다.